

Course No. CSSE 601

Distributed Systems

Course No. CSSE 601

Remote Procedure Call

Objective

- Introduction
- Message Passing
- Remote Procedure Call
- RPC: Basic Operation, Design Issues
- RPC: Structure, Binding, Model, Problems
- RPC: Marshalling, Problems, Approach
- Paradigm for Building Distributed Programs
- RPC Tools

Introduction

- **Communication Primitives:** There are **two models** that are widely accepted to develop **distributed operating system**.
- Message Passing
- Remote Procedure Call

Message Passing

- The Message Passing model provides **two** basic communication Primitives
 - The **Send Primitives** has two parameters:
 - A **message** and its **destination**.
 - The **recv primitive** has also two parameters:
 - The **source of a message** and a **buffer for storing** the message.
 - An application of these primitives can be found in **client server computation model**.

Message Passing

Blocking vs Non Blocking Primitives:

- In the **standard message passing model** messages are copied **three times**
 - From **user buffer** to **kernel buffer**
 - From **kernel buffer on sending computer** to the **kernel buffer on receiving computer**.
 - From **receiving buffer** to **user buffer**.
- With **non- blocking primitive**, the **send primitive** return the control to **user process**.
- While the **Receiving primitive** respond by **signaling** and provide a **buffer to copy**.

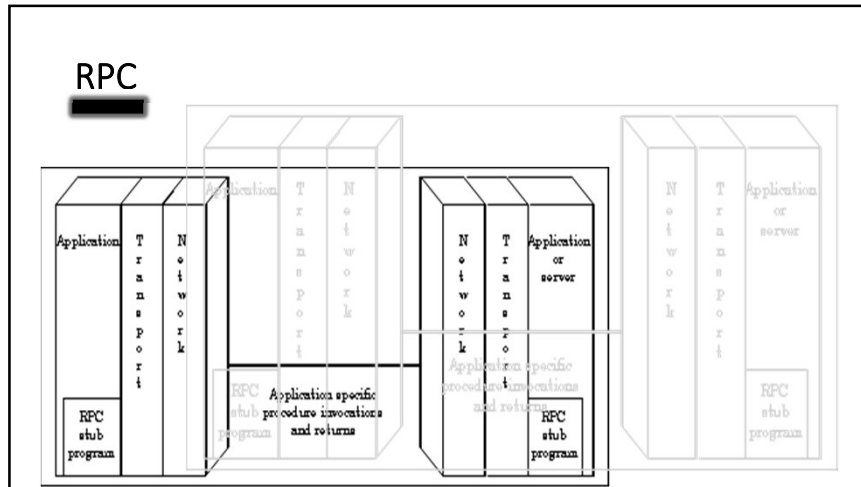
Message Passing

Blocking VS. Non Blocking Primitives:

- A **significant disadvantages** of **non-blocking** is that **programming** becomes **difficult**.
- With **synchronous primitive**, a **send primitive** is **block** until a corresponding **Receive primitive** is **executed** at the **receiving computer**.
- With **asynchronous primitive** ,the messages are **buffered**.
- A **send primitive** is **not block** even if no corresponding **execution** of a **receive primitive**.

Remote Procedural Call

- A **more natural way to communicate** is through **Procedural call**:
 - **every language supports** it .
 - **semantics** are **well defined** and **understood** .
 - **natural** for **programmers** to use.
- **Programmer** using such a **model** must handle the following details:
 - **Pairing of responses** with **request messages**.
 - **Data representation**.
 - **Knowing the address** of **remote machine** on the **server**
 - **Taking care** of **communication** and **system failure**



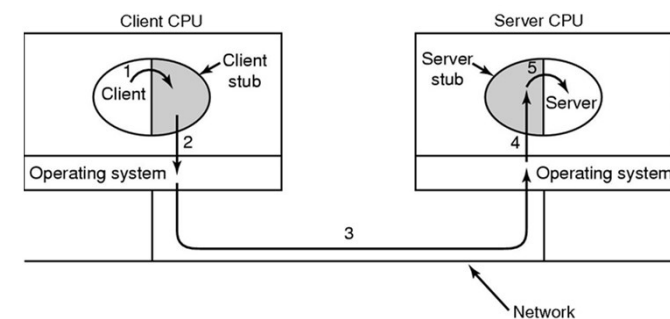
RPC: Basic Operation

- The RPC Mechanism is based on the observation that a procedural call is well known for transfer of control and data within a program running on a single machine.
- On invoking a remote procedure, the calling process is suspended.
- If any parameter are passed to the remote machine where the procedure will execute.
- On completion, the results are passed back from server to client and resuming execution as if it had called a local procedure.

RPC: Design Issues

- RPC mechanism is based on the concept of stub procedures.
- The server writer writes the server and links it with the server-side stubs; the client writes her program and links it with the client-side stub.
- The stubs are responsible for managing all details of the remote communication between client and server.

RPC: Design Issues



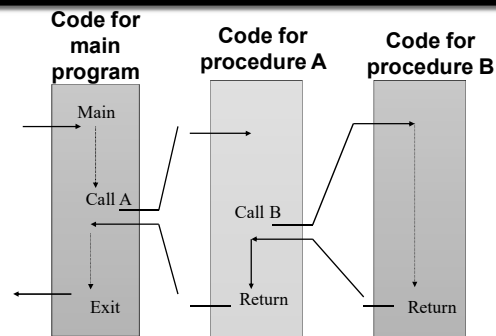
RPC: Structure

- When a program (client) makes a remote procedure call, say $p(x,y)$, it actually makes a local call on a dummy procedure or a client-side stub procedure p .
- The client-side stub procedure constructs a message containing the identity of the remote procedure and parameters and then sends it to the remote machine.
- A stub procedure at the server side receives the message and makes a local call to the procedure specified in the message.

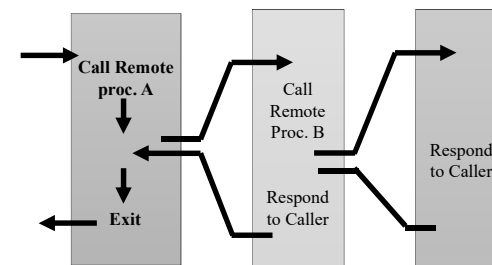
RPC: Structure

- After execution control returns to the server stub procedure, which then returns the control to the client-side stub.
- The stub procedures can be generated at compile time or can be linked at run time.

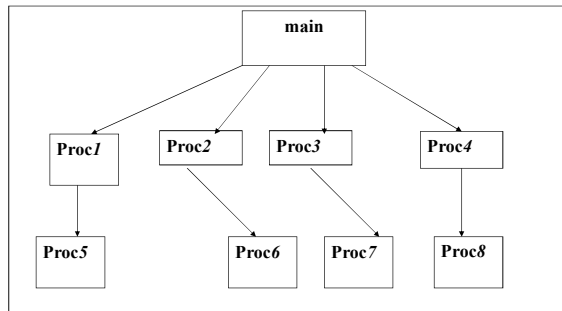
Execution of Conventional Procedure Call



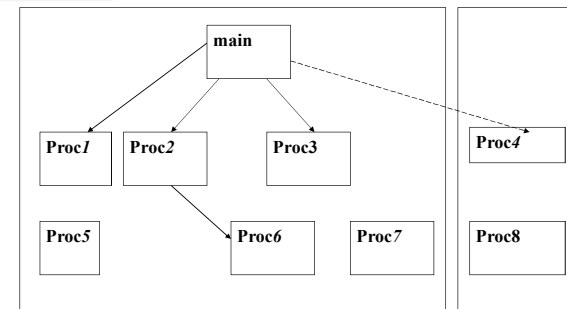
The Procedural Model in Distributed Systems



Conceptual model for Conventional Procedure Calls



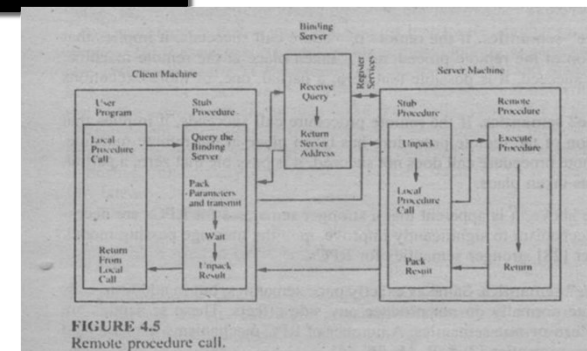
RPC Model



RPC: Binding

- Binding is process that determines the remote procedure, and the machine on which it will be executed.
- It may also check the compatibility of parameters passed and procedure type called.
- Binding server essentially store the server machine along with the services they provide.
- Another approach used for binding is where the client specifies the machine and the service required and the binding server returns the port number for communication.

Parameter and Result



RPC: Problems

- **Procedures** reside on **different machines**
 - This means we cannot simply jump to the start of the procedure
 - We need to use network communication techniques to interact with the remote machine.
- **Procedures** reside in **different address spaces**
 - This means that we cannot pass pointers from caller to callee because a pointer is only valid in one AS
- **Parameters** and results need to be passed across the **network**
- Machine can crash
 - What happens if you call a remote procedure and the remote machine crashes before returning?

Marshalling

- There are two approaches for distributed Application.
- Communication Oriented Design
 - Begins with communication protocol
 - Message format and syntax
 - Design client server component by specifying interaction with incoming and outgoing messages.
- Application Oriented
 - Begin with the Application
 - Build and test a working version that operate on single machine
 - Divide the program into two or more pieces and add communication protocol to allow each piece to execute on separate machine

Marshalling Problems

- The server may implement multiple procedures and it may be necessary for the client to specify which procedure is to be invoked
- Different machines may use different character representations
- Machine may represent integers differently
- Floating point numbers may also be different
- We must therefore send data from one machine to another in some canonical form
- For example, the Sun XDR standard

Marshalling Problems

- Worst of all, any pointer-based data structures cannot be passed because client-side pointers will not be valid on the server
- There are two main solutions to this problem:
 - Forbid pointer-based data structures
 - Encode the data
- When encoding the data, we need to be careful to avoid losing referential integrity

RPC Approach

- The RPC model uses the application oriented approach which emphasizes the problem to be solved instead of the communication needed.
- RPC divides the program at procedure boundaries.
- Splitting into local and remote parts can be made without major modifications to program structure.

Paradigm for Building Distributed Programs

- There are two approaches for distributed Application.
- Communication Oriented Design
- Begins with communication protocol
- Message format and syntax
- Design client sever component by specifying interaction with incoming and outgoing messages.
- Application Oriented
- Begin with the Application
- Build and test a working version that operate on single machine
- Divide the program into two or more pieces and add communication protocol to allow each piece to execute on separate machine

Client/Server and RPC

- Like conventional procedure call, the system, a remote procedure call transfers control to the called procedure.
- The system suspend the execution of calling procedure during the call and only allows the called procedure to execute.
- When a remote program issues a response, it corresponds to the execution of a return in a conventional procedure call.
- Controls flows back to the caller and called procedure ceases to execute.

Client/Server and RPC

- One remote procedure may call another remote procedure.
- Conventional procedures actually accept a few arguments and return only a few result.
- Server can accept or return arbitrary amount of data.
- It would be ideal if local and remote procedural calls behaved identical
 - Network delays
 - Same address space
- Remote procedure call cannot pass pointer as arguments
 - RPC cannot share the Caller's Environment

Middleware And Object Oriented Middleware

- Variety of commercial tools are available that use the RPC paradigm to construct client-server software.
- These tools are generally called middleware because they fit between a conventional application program and the network software.
- The role of middleware is to ease the task of designing, programming and managing distributed applications by providing a simple, consistent and integrated distributed programming environment.

Middleware And Object Oriented Middleware

- Essentially, middleware is a distributed software layer, or 'platform' which abstracts over the complexity and heterogeneity of the underlying distributed environment
- Different middleware platforms support different programming models.
- Perhaps the most popular model is object based middleware in which applications are structured into (potentially distributed) objects that interact via location transparent method invocation.

RPC Tools

- Fortunately, we don't have to write the client and server stub (skeleton) code ourselves
- Instead, we use tools that do most of the work for us
- These tools are known as interface generators or stub generators
 - The most common one is RPCGEN
 - Also, Common Object Request Broker Architecture (CORBA)
- Stub generators use an interface definition language (IDL) to describe the client and server code.

RPC Tools

- **gRPC**
- **Overview:** An open-source RPC framework developed by Google that uses HTTP/2 for transport and Protocol Buffers for serialization.
- **Advantages:** High performance, supports bi-directional streaming, strong typing with Protocol Buffers, and supports multiple programming languages.
- **Use Cases:** Microservices communication, real-time communication, and low-latency applications.

RPC Tools

• **RESTful APIs**

- **Overview:** An architectural style for designing networked applications, relying on standard HTTP methods (GET, POST, PUT, DELETE) and resources identified by URLs.
- **Advantages:** Simplicity, statelessness, wide adoption, and ease of use with standard HTTP clients and tools.
- **Use Cases:** Web services, microservices, and integration with third-party services.

RPC Tools

• **GraphQL**

- **Overview:** A query language for APIs developed by Facebook that allows clients to request exactly the data they need.
- **Advantages:** Flexibility in querying, reduces over-fetching and under-fetching of data, strong type system, and introspection capabilities.
- **Use Cases:** Client-server communication, mobile and web applications, and scenarios where clients need precise control over the data they receive.

RPC Tools

• **Message Brokers** (e.g., Apache Kafka, RabbitMQ)

- **Overview:** Middleware that enables asynchronous communication between services by passing messages through a queue or topic system.
- **Advantages:** Decoupling of services, fault tolerance, scalability, and support for asynchronous communication patterns.
- **Use Cases:** Event-driven architectures, microservices communication, and real-time data streaming.

RPC Tools

• **Service Mesh** (e.g., Istio, Linkerd)

- **Overview:** An infrastructure layer that handles service-to-service communication, providing features like load balancing, service discovery, retries, and monitoring.
- **Advantages:** Enhanced security, observability, control over microservices communication without changing application code, and support for advanced routing.
- **Use Cases:** Large-scale microservices architectures, environments requiring advanced traffic management, and scenarios needing enhanced security and observability.

RPC Tools

• GraphQL

- **Overview:** A query language for APIs and a runtime for executing those queries, developed by Facebook. It allows clients to request exactly the data they need.
- **Advantages:** Reduces over-fetching and under-fetching of data, provides a flexible query structure, and includes built-in introspection capabilities.
- **Use Cases:** API design for front-end and mobile applications, complex data requirements, and scenarios where clients need to control the data they receive.

RPC Tools

• WebRTC

- **Overview:** A free, open-source project providing web browsers and mobile applications with real-time communication via simple application programming interfaces (APIs). It supports audio, video, and data sharing.
- **Advantages:** Real-time peer-to-peer communication, low latency, and support for various media types.
- **Use Cases:** Video conferencing, peer-to-peer file sharing, and real-time communication in web and mobile applications.

RPC Tools

• ZeroMQ

- **Overview:** A high-performance asynchronous messaging library aimed at use in scalable distributed or concurrent applications. It provides a message queue but operates at the socket level.
- **Advantages:** High throughput, low latency, and flexible messaging patterns.
- **Use Cases:** Financial services, high-frequency trading, distributed computation, and scenarios requiring high-speed messaging.

RPC Tools

• Apache Pulsar

- **Overview:** A distributed open-source pub-sub messaging system originally created at Yahoo. It has features like multi-tenancy, persistent storage, and a built-in schema registry.
- **Advantages:** Scalability, support for geo-replication, and low latency.
- **Use Cases:** Event streaming, real-time analytics, and messaging in cloud-native applications.

RPC Tools

- These tools and architectures provide modern alternatives to traditional RPC, each with its strengths and suited for different scenarios in distributed systems. The choice of tool or architecture often depends on specific application requirements, such as performance needs, scalability, ease of use, and the development environment.

Query Regarding Lecture

Email: dr.m.naseem622@gmail.com

Thanks

Email: dr.m.naseem622@gmail.com